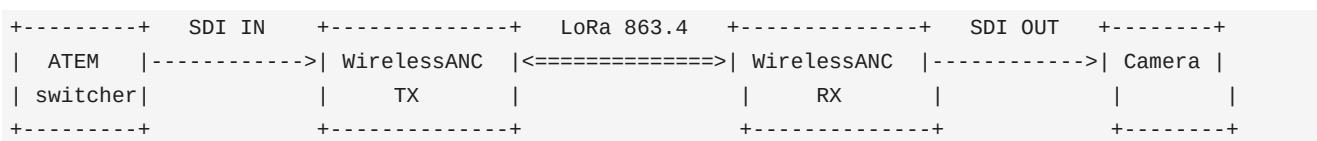


WirelessANC - Architecture & Internals

This document explains what happens behind the scenes in the WirelessANC SDI-over-LoRa gateway.

1. System overview

WirelessANC replaces a physical SDI return path between an ATEM switcher and a remote camera. The switcher embeds tally (on-air / preview flags per camera) and camera control (lens, color, gain, etc.) in the SDI stream. Normally a second SDI cable carries that data to the camera. Here, a TX board reads it from SDI IN and a RX board writes it back onto SDI OUT at the camera end.



There is no TX SDI OUT -> RX SDI IN cable. The only link between boards is LoRa.

Hardware stack (each board)

Layer | Component
 MCU | Arduino Nano Matter (EFR32MG24)
 SDI | Blackmagic 3G-SDI Shield (Serial1 protocol to shield MCU)
 RF | LLCC68 LoRa (SPI: CS=10, DIO1=2, RESET=3, BUSY=9)

Both BMD_SDITallyControl and BMD_SDICameraControl objects call begin() - they share the same shield serial link but expose separate tally and camctrl register banks.

2. Data paths

TX (WirelessANC_TX)

```

loop every 10 ms:
  1. pollMonitorSerial()      - Enter toggles debug logging
  2. if camctrl available:
      read ICDATA bundle from shield
      if changed -> LoRa TX type 0x01 (raw bundle)
  3. if tally available:
      read ITDATA from shield
      if changed -> compress -> LoRa TX type 0x03
  4. pollAirtimeReport()      - always prints every 10 s
  
```

SDI override on TX: setOverride(false) - the board passes through what arrives on SDI IN without replacing it.

RX (WirelessANC_RX)

```

loop:
  
```

```

1. pollMonitorSerial()
2. on LoRa RX interrupt:
    read packet -> handle by type
    tally -> decompress -> sdiTallyControl.write()
    camctrl -> sdiCameraControl.write(..., forceWrite=true)
3. every 250 ms: re-inject last tally (keeps camera tally LEDs fresh)

```

SDI override on RX: setOverride(true) - injected data replaces empty/pass-through SDI on SDI OUT. Persistent override is required; momentary override caused camctrl inject failures in earlier builds.

3. LoRa framing

Each over-the-air packet is:

```
[type: 1 byte][payload: N bytes]
```

Maximum raw payload: 254 bytes (255-byte RadioLib buffer minus type byte).

Type	Name	Direction	Content
0x01	CAMCTRL	TX -> RX	Full raw ICDATA bundle from shield (concatenated BMD SDI packets)
0x02	TALLY_RAW	either	Uncompressed tally bytes (legacy / fallback)
0x03	TALLY_COMP	TX -> RX	Compressed tally (default on TX)
0x04	CAMCTRL_BUNDLE	-	Structured multi-packet encode (codec support, not used on air today)
0x05	CAMCTRL_RAW	RX	Alias for raw camctrl inject

RF parameters

Parameter	Value
Frequency	863.4 MHz
Bandwidth	500 kHz
Spreading factor	5

TX tracks airtime via `lora.getTimeOnAir()` and prints utilization every 10 seconds.

Deduplication (TX)

- Tally: only transmitted when SDI tally bytes change (memcmp vs last read).
- Camctrl: only transmitted when the full ICDATA bundle changes.

This avoids saturating LoRa when the ATEM sends identical bundles repeatedly.

4. BMD SDI camera control packet format

Camera control on SDI uses concatenated packets:

```
[dest][payloadLen][cmd][reserved] + payload + pad to 4 bytes
```

- payloadLen = bytes after the 4-byte header (excludes padding).
- Total packet size = 4 + payloadLen, rounded up to a multiple of 4.

Example (12 bytes):

```
01 06 00 00 00 00 80 01 48 00 00 00
|  |  |  |  +- payload (6 bytes) -+ +- pad -+
|  |  cmd=0
|  len=6
dest=cam 1
```

The TX reads a bundle of back-to-back packets from the shield ICDATA register. The parser in CamctrlCodec.h walks the bundle packet-by-packet for debug logging.

ICLENGTH quirk (padding recovery)

The shield ILENGTH register is 8 bits and sometimes reports one byte less than the actual data in ICDATA - typically the final padding byte of the last packet in a bundle.

BMDSDICameraControl::read() (patched in this project):

1. Read regRead8(ICLENGTH) bytes from ICDATA.
2. If the last packet is exactly one byte short of its padded length, read one more byte from ICDATA + length.
3. Arm the incoming bank (ICARM).

Without this fix, a 240-byte bundle was read as 239 bytes and the last packet failed to parse.

5. Tally compression

Tally SDI data is one byte per camera with 2-bit flags (PGM, PVW, etc.). Most bytes are zero or low-entropy.

Compressed format (TallyCodec.h, LoRa type 0x03):

```
[sdiLen: 1 byte][packed: ceil(sdiLen/4) bytes]
```

Each camera's 2-bit flags are packed 4 cameras per byte. RX decompresses back to the original SDI tally length before sdiTallyControl.write().

Typical savings: ~75% vs raw tally for multi-camera tally maps.

6. Shield initialization

BmdShieldInit.h runs up to 5 attempts:

```
tally.begin();
cam.begin();
```

Both must report shieldInitialized. Boot messages are gated behind debug logging; shield init FAILED always prints.

Startup delay: 500 ms after Serial.begin() before shield/LoRa init (BoardConfig.h).

7. Serial logging

Output | When

READY wirelessanc-v1 TX/RX | Always at boot

AIRTIME ... | Always on TX, every 10 s

ERROR ... | Always (LoRa/shield failures)

Packet hex, camctrl parse, LoRa dumps | Only when logging ON

Toggle: press Enter on USB serial (115200 baud). Firmware prints [monitor] serial ON or OFF.

The Python monitor (scripts/monitor.py) mirrors this - display starts OFF; Enter toggles both local display and sends Enter to the board.

8. Build & flash pipeline

```
make compile-tx/rx -> arduino-cli compile (FQBN: nano_matter, protocol_stack=none)
make upload-tx/rx  -> scripts/flash_board.py (OpenOCD with adapter serial filter)
                   -> scripts/verify_serial.py (waits for READY line with build ID)
```

When two boards are plugged in, plain arduino-cli upload targets the first CMSIS-DAP device. flash_board.py maps /dev/cu.usbmodem* to adapter serial so each board gets the correct hex.

9. Source file reference

File | Purpose

WirelessANC_TX/WirelessANC_TX.ino | TX main loop

WirelessANC_RX/WirelessANC_RX.ino | RX main loop

include/BoardConfig.h | Timing, LoRa pins, RF constants

include/CamctrlCodec.h | BMD packet parse, bundle walk, padding detect

include/CamctrlLog.h | Debug logging for camctrl bundles

include/TallyCodec.h | Tally compress/decompress

include/MonitorRuntime.h | Enter-to-toggle, MONITOR_GUARD()

include/MonitorLog.h | Structured debug log helpers

include/BmdShieldInit.h | Shield begin() retry

libraries/.../BMDSDICameraControl.cpp | Patched read() with padding recovery

10. Operational notes

- Gain / color / lens changes from ATEM Software Control travel as camctrl bundles (type 0x01).
- Tally is latency-sensitive; RX re-injects every 250 ms so camera tally LEDs stay lit even between LoRa updates.
- LED on each board blinks LOW during active LoRa TX/RX.
- If TX/RX ports are swapped, run make identify-ports and update TX_PORT / RX_PORT in the Makefile.